

09

MÓDULO DE GERENCIAMENTO DE POUSO E ESTABILIZAÇÃO DE BASE

Relatório técnico do MGPEB para a missão Aurora Siger: cenário de pouso, regras de decisão, modelagem, contexto histórico e governança da base.

CENÁRIO DE POUSO E MÓDULOS DA COLÔNIA

Grupo 9

Felipe de Barros Franco · felipedebarrasfranco@gmail.com

Guilherme Santos · guilhermesantos3828@gmail.com

Código-fonte do protótipo: github.com/Guilherme-AVVO/Aurora-Siger-MGPEB

A frota da missão Aurora Siger chega à órbita de Marte em módulos separados. Cada módulo é um veículo com combustível e motor próprios e leva uma parte da colônia. O MGPEB funciona como uma torre de controle: não pilota os módulos, decide quem pousa, em que ordem e em qual área. A decisão é tomada com o módulo ainda em órbita, porque depois que a descida começa não dá para voltar. Como o atraso de comunicação entre Terra e Marte pode chegar a 22 minutos [F1], o sistema decide sozinho.

Definimos nove módulos. Cada um tem nome, tipo de carga, criticidade (1 a 5, o quanto perdê-lo seria grave), prioridade de pouso (1 a 5), massa, combustível, carga da bateria e estado dos sensores. Prioridade e criticidade são separadas de propósito. A Logística Alfa tem criticidade 1 e prioridade 5: ela desce antes da Energia e funciona como precursora, testando a área antes dos módulos críticos. O horário de chegada à órbita é representado pela posição no cadastro e desempata módulos de mesma prioridade.

TABELA 1 - MÓDULOS CADASTRADOS (NA ORDEM DA FILA, DEPOIS DA ORDENAÇÃO)

Módulo	Criticidade	Prioridade	Massa (t)	Combustível (kg)	Energia	Sensores
Logística Alfa	1	5	12	1800	85%	ok
Energia	5	5	25	3300	90%	ok
Suporte de Vida	5	4	22	2900	80%	ok
Habitação	4	4	30	3800	75%	ok
Comunicação	3	3	10	1500	35%	ok
Médico	4	3	15	1600	70%	ok
Laboratório	2	2	18	2400	80%	falha
Logística Beta	2	2	14	2000	85%	ok
Extração de Gelo	2	1	20	2600	80%	ok

Os valores são fictícios e foram escolhidos para que a demonstração passe por todos os caminhos de decisão. Em negrito estão os três módulos que o sistema deve barrar: Comunicação pela energia, Médico pelo combustível e Laboratório pelo sensor. As massas, entre 10 e 30 t, ficam na faixa que a NASA projeta para pousos de missões humanas [R5].

A operação usa três estruturas lineares. A fila de pouso recebe os módulos já ordenados por prioridade. Três listas guardam os destinos (pousados, em espera e em alerta) e uma pilha registra cada decisão. O detalhe de cada uma está no anexo.

TABELA 2 - LOCAIS DE POUSO CANDIDATOS

Local	Latitude	Elevação	Rajada	Vagas	Tempestade	Recurso local
LOC-A Planície	8°N	-3,2 km	22 m/s	8 + 2	5%	água distante
LOC-B Cratera	22°S	-1,8 km	28 m/s	7 + 2	10%	minerais hidratados
LOC-C Vale Boreal	28°N	-2,5 km	26 m/s	9 + 0	15%	gelo de água

Os limites vêm de requisitos escritos no padrão da NASA [R1] a partir das restrições usadas na escolha do local do rover Curiosity [R3]: latitude entre 30°S e 30°N, elevação até -1 km, rajadas abaixo de 30 m/s e uma área alternativa para emergências. O LOC-C tem água, mas não tem área alternativa e fica perto do limite em latitude, rochas e albedo. Por isso a equipe recomenda o LOC-A como local primário; o LOC-C só entraria com uma dispensa formal registrada, como a NASA faz quando aceita seguir sem um requisito [R2]. As vagas (área primária + alternativa) representam quantos módulos cabem em cada área mantendo distância de segurança, já que os motores levantam detritos que podem atingir quem já pousou [R6].

REGRAS DE DECISÃO E PORTAS LÓGICAS

Cada módulo retirado da fila passa por quatro verificações. Cada uma vira uma variável booleana:

- **S** (sensores e energia): sensores funcionando e bateria em pelo menos 40%;
- **C** (combustível): combustível maior ou igual a massa × 110 kg;
- **T** (atmosfera): sem tempestade de poeira, sorteada por módulo com a chance do local;
- **A** (área): vaga na área primária ou, se ela estiver cheia, na alternativa.

$$AUTORIZA = S \cdot C \cdot T \cdot A \mid ALERTA = \neg(S \cdot C) \mid ESPERA = S \cdot C \cdot \neg(T \cdot A)$$

Falha em S ou C manda o módulo para alerta, porque o problema é do próprio módulo e esperar não resolve. Falha em T ou A manda para espera: a tempestade passa e uma vaga pode abrir. Na forma de portas, ALERTA é uma NAND de S e C, e ESPERA é uma NOR entre ALERTA e $(T \cdot A)$. As três saídas nunca valem 1 ao mesmo tempo.

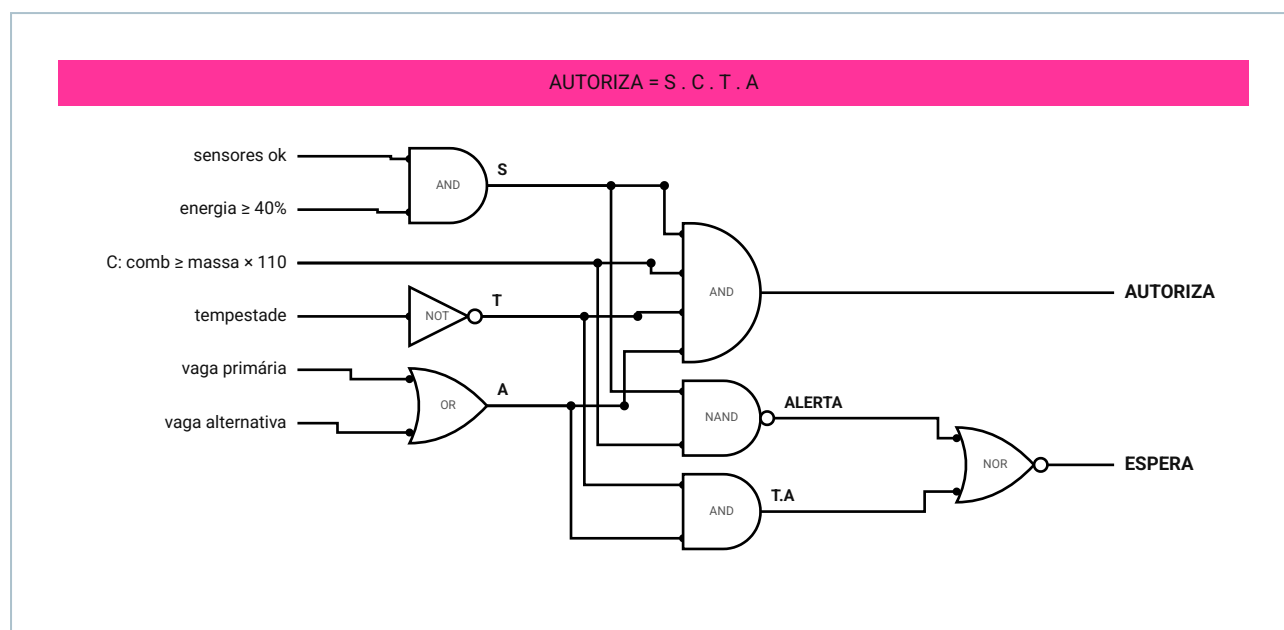


Figura 1 - Circuito de decisão do MGPEB: autorização, alerta (NAND) e espera (NOR)

Fonte: Elaborado pelos autores

Com quatro variáveis há 16 combinações. A tabela resume todas, usando "x" quando o valor não muda o resultado. Como o programa para no primeiro portão que falha, a última coluna é também o motivo registrado.

TABELA 3 - TABELA-VERDADE RESUMIDA DAS 16 COMBINAÇÕES

S	C	T	A	Linhas	Destino	Motivo registrado
0	x	x	x	8	ALERTA	falha de sensor ou energia abaixo do mínimo
1	0	x	x	4	ALERTA	combustível insuficiente
1	1	0	x	2	ESPERA	tempestade de poeira
1	1	1	0	1	ESPERA	sem vaga nas áreas de pouso
1	1	1	1	1	AUTORIZADO	área primária ou alternativa

No código, as expressões viram uma sequência de `if/elif/else` na ordem S, C, T e A, com `continue` em cada falha. O resultado é o mesmo em qualquer ordem, mas essa escolha tem dois motivos. Um módulo com defeito não precisa de previsão do tempo, então o sorteio só acontece para quem passou em S e C. E o portão A fica por último porque é o único que altera dados: ele ocupa uma vaga, e só faz isso depois de todos os outros testes. O vento não vira verificação porque as rajadas máximas dos três locais já ficam abaixo de 30 m/s; esse requisito é atendido na escolha do local.

CÓDIGO 1 - PORTÃO C EM AVALICAO_MODULOS.PY

PYTHON

```

1  def av_combustivel(modulo):
2      comb = modulo["comb"]
3      massa = modulo["massa"]
4      if massa * 110 > comb:
5          return False, "combustível insuficiente"
6      else:
7          return True, None

```

PROTÓTIPO EM PYTHON

O protótipo está no repositório do projeto (github.com/Guilherme-AVVO/Aurora-Siger-MGPEB) e usa só a biblioteca padrão. O arquivo `main.py` cuida do fluxo e `avalicao_modulos.py` guarda os quatro portões. O programa pergunta o local de pouso, mostra o cadastro, ordena os módulos com bubble sort, processa a fila e mostra o relatório final, o histórico e um menu de consultas com três buscas lineares (menor combustível, maior prioridade e tipo de carga). O sorteio do clima usa `random.seed(42)` para a demonstração sair igual em toda execução.

TABELA 4 - RESULTADO DA EXECUÇÃO NOS TRÊS LOCAIS

Local	Pousados	Em espera (tempestade)	Em alerta
LOC-A	5 de 9	Suporte de Vida	Comunicação, Médico, Laboratório
LOC-B	5 de 9	Suporte de Vida	Comunicação, Médico, Laboratório
LOC-C	4 de 9	Energia, Suporte de Vida	Comunicação, Médico, Laboratório

No LOC-C a Energia, que tem prioridade e criticidade máximas, fica em órbita por causa do clima. Nenhum módulo passa por cima das regras de segurança. Com as vagas atuais ninguém precisa da área alternativa; esse caminho e a espera por falta de vaga foram testados reduzindo as vagas do LOC-B para 2 + 2.

MODELAGEM DE FUNÇÕES APLICADAS

COMBUSTÍVEL MÍNIMO PARA POUSAR (FUNÇÃO LINEAR)

$$C_{\min}(m) = a \cdot m, \text{ com } a = \beta \cdot (1 + r) = 100 \times 1,10 = 110 \text{ kg/t}$$

A massa m (toneladas) é a variável independente e $C_{\min}$ (kg) a dependente. $\beta = 100$ kg/t é o combustível gasto para frear cada tonelada na descida e $r = 10\%$ é a reserva de segurança. A reserva existe porque relevo e vento mudam o consumo durante a descida motorizada [R3]. Os dois valores são premissas da equipe. É uma função linear [F6]: o coeficiente linear é zero, então a reta passa pela origem, e o coeficiente angular $a = 110$ diz que cada tonelada a mais exige sempre mais 110 kg.

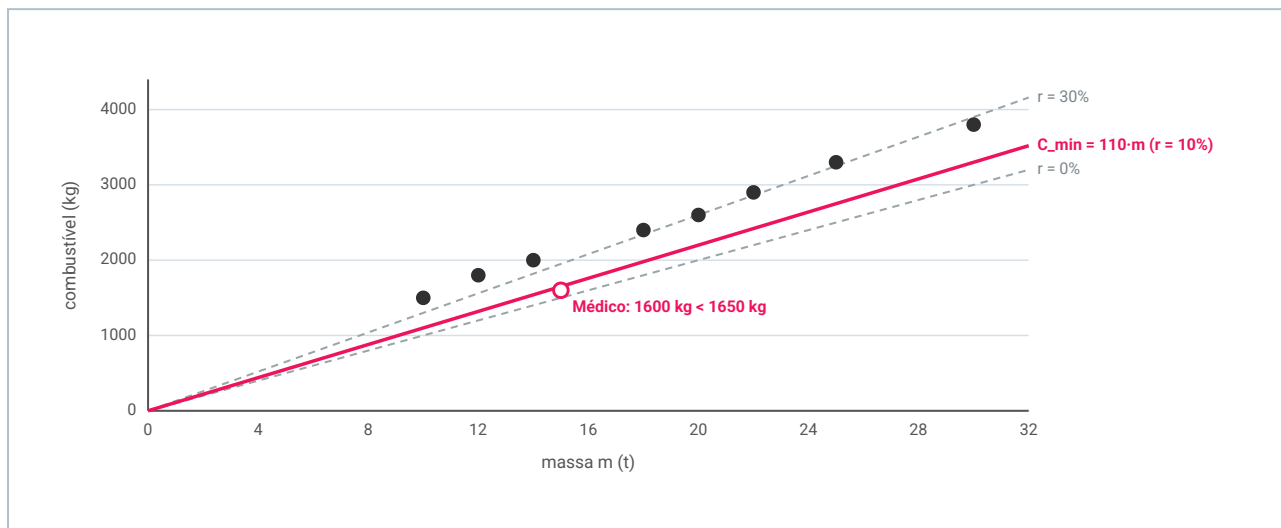


Figura 2 - Reta do combustível mínimo e os nove módulos (pontos)

Fonte: Elaborado pelos autores

Quem fica acima da reta passa no portão C. O Médico é o único ponto abaixo: com 15 t, precisa de 1650 kg e tem 1600. A distância vertical até a reta é a margem que o programa imprime. Aumentar a reserva deixa a reta mais inclinada. Com $r = 0\%$ nenhum módulo é barrado, com 20% continua só o Médico e com 30% a Habitação também cai. A reserva de 10% é a escolha entre segurança e número de módulos que conseguem descer.

RISCO DE TEMPESTADE AO MANDAR VÁRIOS MÓDULOS (FUNÇÃO EXPONENCIAL)

$$P(n) = 1 - (1 - p)^n$$

n é quantos módulos tentam pousar e p a chance de tempestade por módulo no local (5, 10 ou 15%). $P(n)$ é a chance de pelo menos um ser adiado. O termo $(1 - p)^n$ é uma exponencial de base entre 0 e 1, portanto decrescente [F6]. A conta $1 - (1 - p)^n$ aplica uma reflexão no eixo x e uma translação vertical. No código, cada módulo que chega ao portão T faz um sorteio independente com essa chance, que é a hipótese do modelo.

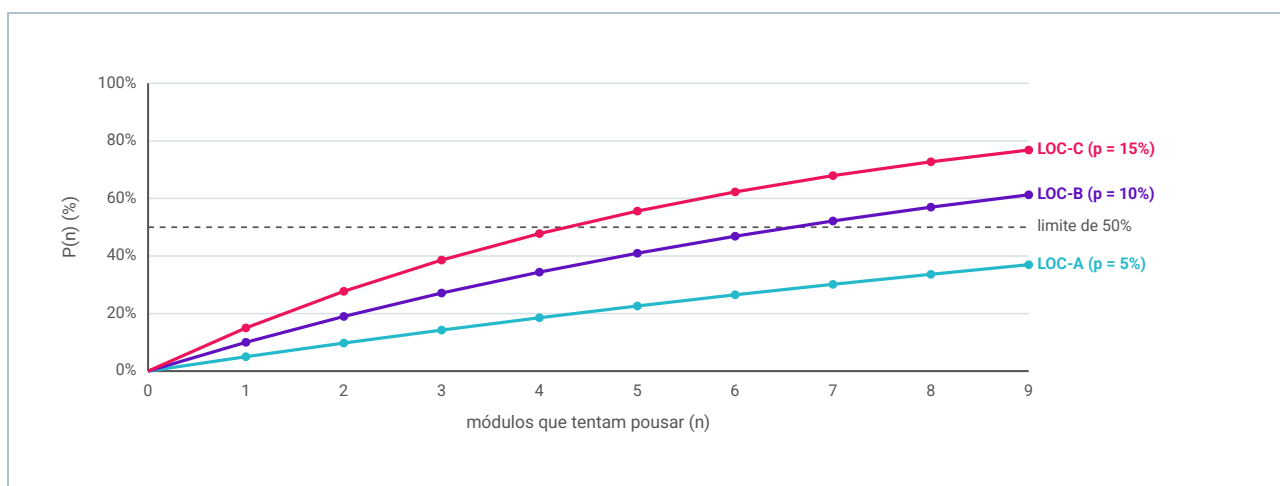


Figura 3 - Chance de pelo menos um adiamento por tempestade em cada local

Fonte: Elaborado pelos autores

A curva sai de zero, sobe rápido e depois desacelera, chegando perto de 100% sem nunca atingir. Quanto maior o p do local, mais rápido ela sobe. Usando a inversa, o logaritmo, dá para calcular quantos módulos podem tentar pousar com risco abaixo de 50%: $n = \ln(0,5) / \ln(1 - p)$. O resultado é 13 módulos no LOC-A, 6 no LOC-B e 4 no LOC-C. Isso responde ao exemplo do enunciado sobre limitar quantos módulos pousam de uma vez. No LOC-C, os seis módulos aptos da frota já passam de 60% de risco, e a execução confirmou: dois deles caíram em tempestade.

CONTEXTO HISTÓRICO E LIMITAÇÕES DE HARDWARE

O ENIAC, construído entre 1943 e 1946, pesava cerca de 30 toneladas, consumia 140 kW e tinha 17.840 válvulas [F7]. Um computador assim nunca caberia num módulo de pouso. O caminho até a computação embarcada passou pelo transistor, menor e mais econômico, pelo circuito integrado e pela integração em escala muito grande (VLSI). A Lei de Moore, que o material da disciplina descreve como a duplicação do número de transistores a cada 18 meses [F7], resume essa miniaturização. Foi ela que permitiu levar um computador a bordo de cada veículo.

Mesmo assim, o hardware que vai a Marte é modesto. O rover Perseverance usa um processador RAD750 de até 200 MHz, com 256 MB de memória RAM e 2 GB de memória flash. O processador é resistente à radiação e o rover leva dois computadores idênticos, para sempre haver um reserva [R7]. Pouca memória, pouco processamento, energia contada e radiação são as limitações típicas desse ambiente. O material da missão acrescenta que painéis solares perdem eficiência em tempestades de poeira e que os computadores consomem energia e geram calor [F1].

Essas limitações aparecem nas escolhas do protótipo:

TABELA 5 - ESCOLHAS DO PROTÓTIPO E A LIMITAÇÃO QUE ELAS CONSIDERAM

Escolha	Por que combina com hardware limitado
Listas, fila e pilha simples	pouca memória e comportamento previsível
Fila como cópia rasa (<code>módulos[:]</code>)	copia só as referências; os módulos não são duplicados

Escolha	Por que combina com hardware limitado
Busca linear, $O(n)$	sem memória extra e sem exigir dados ordenados
Bubble sort, $O(n^2)$	simples, estável e sem memória extra: 64 comparações e 19 trocas para 9 módulos
Regras fixas em <code>if/elif/else</code>	a mesma entrada sempre dá a mesma decisão, o que facilita testar
Decisão autônoma e histórico	não depende da Terra [F1] e deixa cada decisão auditável [R2]

Em sistemas embarcados, um comportamento previsível costuma valer mais que a maior velocidade possível. Um algoritmo simples e determinístico é mais fácil de testar e de certificar do que um mais rápido e mais complexo, e com poucos dados a diferença de tempo nem aparece.

ESG E GOVERNANÇA NA BASE AURORA SIGER

ESCOLHA DA ÁREA DE POUSO E IMPACTO AMBIENTAL

Os requisitos de local (latitude, elevação, declive, rochas, poeira e área alternativa) já são critérios de escolha. Do lado ambiental, os motores de pouso erodem o solo e lançam detritos [R5]. Concentrar os pousos numa área delimitada, com distância entre módulos, limita a região alterada. Os estudos da NASA para missões humanas também preveem um único local de pouso usado por várias tripulações [R6], o que vai na mesma direção. O caso do LOC-C mostra a troca que a governança precisa registrar: é o único local com água no subsolo, mas pede uma dispensa de requisito de segurança.

ENERGIA, RESÍDUOS E RECURSOS LOCAIS

O portão S barra módulos com menos de 40% de bateria, e o módulo de energia tem prioridade máxima porque habitação, laboratório e área médica dependem dele. Gelo e minerais hidratados nos locais candidatos permitem produzir água na própria base e reduzir a dependência de suprimentos da Terra. Locais perto do equador recebem mais luz solar, outro ponto a favor do LOC-A. Para resíduos, o mesmo princípio de registro vale: o que entra e o que sai da base precisa ser medido para ser gerido, alinhado à Agenda 2030 e aos relatórios no padrão GRI estudados na disciplina [F8].

TRANSPARÊNCIA E PARTICIPAÇÃO

O MGPEB registra cada decisão com o motivo, na pilha de histórico, então qualquer pessoa da colônia pode conferir por que um módulo foi barrado. As regras são as mesmas para todos: o Médico, que é crítico, vai para alerta por falta de combustível, e a Energia fica em espera no LOC-C. Dispensas de requisitos seguem o modelo da NASA, com aprovação formal e justificativa escrita [R2]. Mandar um módulo pouco crítico como precursor é uma decisão sobre quem assume o risco primeiro; com tripulação a bordo, ela teria de ser discutida abertamente. Por fim, toda afirmação de sustentabilidade da base deve vir acompanhada da regra ou do dado que a sustenta, para não cair no greenwashing que o material da disciplina descreve [F8].

ANEXO

ESTRUTURAS DE DADOS

TABELA 6 - ESTRUTURAS LINEARES USADAS NO PROTÓTIPO

Estrutura	Variável	Entra por	Sai por	Uso
Lista	<code>modulos</code>	cadastro	não sai	cadastro e base das buscas
Listas	<code>pousados</code> , <code>alerta</code> , <code>em_espera</code>	<code>append</code>	não sai	destino de cada módulo
Fila (FIFO)	<code>av_modulos</code>	cópia ordenada	<code>pop(0)</code>	fila de pouso
Pilha (LIFO)	<code>historico</code>	<code>append</code> (topo)	<code>pop()</code> (topo)	registro de decisões

Lista. Cada módulo é um dicionário com os mesmos oito campos, guardado na lista `modulos`, por exemplo `{"nome": "Médico", "prio": 3, "massa": 15, "comb": 1600, ...}`. O bubble sort ordena a lista no próprio lugar por prioridade decrescente. Como compara com `<` estrito, nunca troca módulos de mesma prioridade, e o empate respeita a ordem do cadastro. Num teste com `<=`, três pares empatados saíram invertidos.

Fila. A fila é criada com `av_modulos = modulos[:]`. A cada volta do laço o primeiro módulo é avaliado e sai com `pop(0)`; o laço roda enquanto a fila tiver itens. A cópia foi necessária porque `av_modulos = modulos` só cria um segundo nome para a mesma lista, e o `pop` esvaziava o cadastro que as buscas usam depois. No LOC-C, a Logística Alfa sai primeiro e pouso; em seguida saem Energia e Suporte de Vida, que vão para espera, e assim por diante até a Extração de Gelo, com a fila vazia no fim.

Pilha. Cada decisão é empilhada com `append`, por exemplo `{"nome": "Médico", "resultado": "ALERTA", "detalhe": "combustível insuficiente"}`. No fim, o histórico é mostrado desempilhando com `pop()`, que sempre tira o último registro. A pilha guarda as nove decisões, inclusive os pousos, e por isso mostra a operação inteira do mais recente para o mais antigo: "9. Extração de Gelo -> AUTORIZADO | primária" sai primeiro e "1. Logística Alfa -> AUTORIZADO | primária" sai por último.

Buscas. As três consultas percorrem `modulos` do começo ao fim. A de menor combustível retorna a Comunicação (1500 kg). A de maior prioridade retorna a Logística Alfa, porque o empate com a Energia fica com o primeiro encontrado. A busca por carga compara em minúsculas, então "logistica" encontra a Logística Alfa e a Logística Beta.

REFERÊNCIAS

- [F1] FIAP. *Ignition Zero: o início da Aurora*. Ciência da Computação, Fase 2, Cap. 1. São Paulo: FIAP, 2025.
- [F2] FIAP. *A programação procedural governando cada ajuste de descida* (portas lógicas e funções). Fase 2, Cap. 2. São Paulo: FIAP, 2025.
- [F3] FIAP. *As estruturas lógicas que refinam a correção de trajetória*. Fase 2, Cap. 3. São Paulo: FIAP, 2025.
- [F4] FIAP. *As estruturas lineares organizando o fluxo de dados da aproximação*. Fase 2, Cap. 4. São Paulo: FIAP, 2025.
- [F5] FIAP. *Os algoritmos que selecionam caminhos e ordenam prioridades de voo*. Fase 2, Cap. 5. São Paulo: FIAP, 2025.
- [F6] FIAP. *As funções aplicadas interpretando as variáveis da atmosfera marciana*. Fase 2, Cap. 6. São Paulo: FIAP, 2025.
- [F7] FIAP. *A evolução computacional que sustenta os sistemas da nave*. Fase 2, Cap. 7. São Paulo: FIAP, 2025.
- [F8] FIAP. *Governança ambiental, social e corporativa (ESG)*. Fase 2. São Paulo: FIAP, 2025.
- [R1] NASA. *Systems Engineering Handbook*, Appendix C: How to Write a Good Requirement. Disponível em: nasa.gov. Acesso em: 3 out. 2026.
- [R2] NASA. *Systems Engineering Handbook*, 5.3 Product Verification. Disponível em: nasa.gov. Acesso em: 3 out. 2026.
- [R3] JET PROPULSION LABORATORY. *MSL Landing Site Selection: User's Guide to Engineering Constraints*, v4.5. Pasadena: JPL, 2007. Disponível em: marsoweb.nas.nasa.gov.
- [R4] NASA KENNEDY SPACE CENTER. *Landing the Space Shuttle Orbiter*. NASA Facts FS-2005-08-026-KSC, 2006. Disponível em: nasa.gov.
- [R5] NASA. *Mars Entry, Descent, and Landing: Challenges for Human Missions*. Moon to Mars Architecture, 2024. Disponível em: nasa.gov.
- [R6] BUSSEY, B.; HOFFMAN, S. J. *Human Mars Landing Site and Impacts on Mars Surface Operations*. IEEE Aerospace Conference, 2016. Disponível em: ntrs.nasa.gov.
- [R7] NASA SCIENCE. *Perseverance Rover Components*. Disponível em: science.nasa.gov. Acesso em: 10 out. 2026.
- Repositório do projeto: github.com/Guilherme-AVVO/Aurora-Siger-MGPEB.